PART II — BUILDING THE INGESTION PIPELINE

Chapter 5

Data Loading From Multiple Sources

"Every pipeline begins with a loader. Every failure begins with not knowing which loader to use."

Chapter 5 — Data Loading from Multiple Sources

Chapter 4 gave you the Document object — the atomic unit of the entire ingestion pipeline. This chapter fills that object with content. Real-world knowledge bases are never tidy. Your data will live in PDFs on a shared drive, text files exported from a CMS, Excel spreadsheets prepared by an analyst, scanned contracts sitting in an archive, and HTML pages scraped from an internal wiki — all at the same time, all requiring a different parsing strategy. LangChain's loader ecosystem exists precisely to absorb that diversity and flatten every source format into the same uniform output: a list of Document objects.

This chapter works through each major loader family in turn. Section 5.1 starts with the simplest case — plain text — and uses it to establish the mental model that carries through the rest of the chapter. Sections 5.2 and 5.3 cover PDFs and directory-level loading, where most of the practical complexity lives. Section 5.4 introduces the UnstructuredLoader as a high-coverage fallback for formats that resist specialised treatment. Sections 5.5 and 5.6 address tabular and semi-structured formats — CSV, Excel, HTML, Markdown, DOCX, and JSON. Section 5.7 tackles the hardest case: broken and scanned PDFs that require OCR. The chapter closes, in Section 5.8, by assembling all the pieces into a single unified file-discovery helper that can walk an arbitrary set of directories, detect the right loader for each file, and hand off a clean document list to the ingestion pipeline.

By the end of this chapter you will be able to load any file your data science team is likely to hand you, handle the edge cases that make junior engineers reach for manual copy-paste, and understand exactly why each loader was designed the way it was — which means you will also know when to deviate from the defaults.

5.1 Text Files with TextLoader

TextLoader is the loader you will use least — and understand most. That is not a contradiction. Plain text files have no structure to parse, no encoding ambiguity to resolve beyond the character set, and no layout to reconstruct. The loader's simplicity makes every design decision explicit, and those decisions recur in every more complex loader you will meet in this chapter.

> **Definition — TextLoader**
> A LangChain loader that reads the entire contents of a plain-text file — .txt, .log, .md, or any extension you point it at — into a single Document object. The page_content field receives the raw string; the metadata field receives at minimum the source key set to the absolute or relative file path. No further parsing, splitting, or structure detection occurs.

Using it is three lines:

```
from langchain_community.document_loaders import TextLoader

loader = TextLoader("../data/textfiles/release_notes.txt", encoding="utf-8")
docs = loader.load()

print(len(docs))          # 1 — always exactly one Document per file
print(docs[0].metadata)   # {"source": "../data/textfiles/release_notes.txt"}
```

The encoding parameter matters

TextLoader defaults to the operating system's locale encoding — typically UTF-8 on Linux and macOS, but CP1252 on Windows. A file exported from Excel on Windows and then loaded on a Linux CI server will raise UnicodeDecodeError unless you specify the encoding explicitly. The safe habit is to always pass encoding="utf-8" and to export your text files with UTF-8 encoding from the start. If you are dealing with legacy files whose encoding you do not know, the chardet library can infer it from the byte pattern.

> **Common pitfall — One document, no page numbers**
>
> TextLoader produces exactly one Document per file, regardless of how long the file is. If you load a 400-page novel as a single text file, the metadata will contain no page field and the page_content will be a single 800,000-character string. The chunker in Chapter 7 will split it correctly — but you will lose the ability to cite a page number, because that information was never in the file. For long documents where page attribution matters, prefer PDF sources, which preserve pagination.

The glob pattern *.txt is the natural companion of TextLoader when used inside DirectoryLoader (see Section 5.3). The combination covers the overwhelming majority of plain-text corpora in a single expression.

5.2 PDFs with PyPDFLoader vs PyMuPDFLoader — Speed and Fidelity Trade-offs

PDF is the format that causes the most ingestion failures in production RAG systems. The reason is not that PDFs are rare — they are everywhere — but that the PDF specification is extraordinarily permissive. A file whose extension is .pdf might contain machine-readable text, or rasterised page images with no text layer at all, or a mixture of both. It might use non-standard font encodings that cause extractors to produce garbage characters. It might embed tables as visual layouts with no semantic column structure. LangChain provides two specialised PDF loaders, and knowing their differences is the single most useful piece of PDF knowledge in this book.

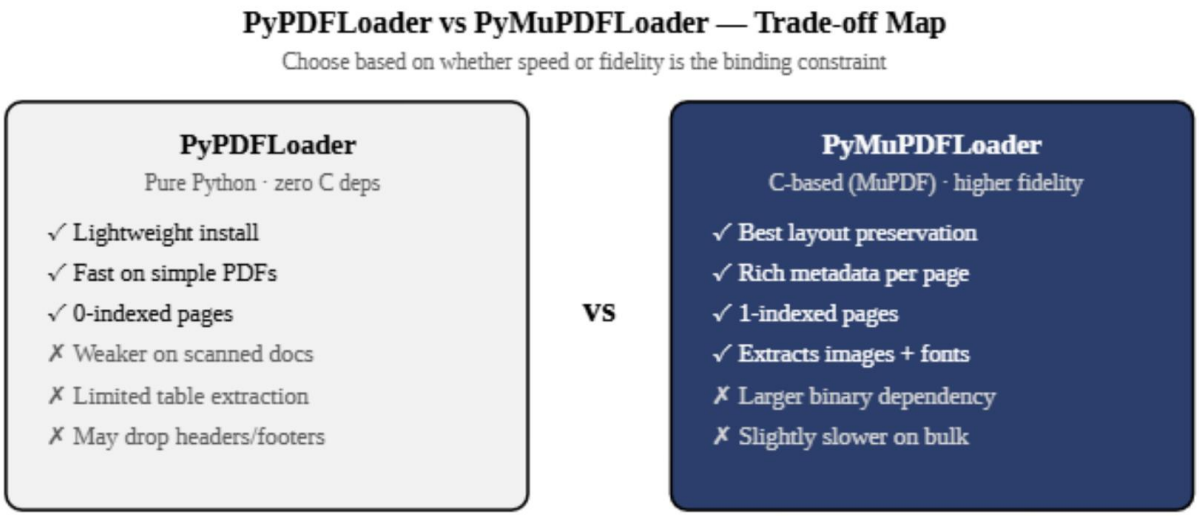

Figure 5.1 — PyPDFLoader vs PyMuPDFLoader trade-off map. The right choice depends on whether your binding constraint is install footprint, parse speed, or extraction fidelity.

PyPDFLoader — pure Python, zero native dependencies

PyPDFLoader is built on the pypdf library, which is written entirely in Python. It installs without compiling any C extension, which makes it safe in restricted deployment environments — Docker images with a minimal base layer, AWS Lambda functions, CI pipelines that prohibit native builds. Its trade-off is fidelity: pypdf reads the PDF's text stream directly, which works well for clean, digitally produced PDFs but degrades on complex layouts, two-column academic papers, and PDFs with custom font encodings.

```
from langchain_community.document_loaders import PyPDFLoader

loader = PyPDFLoader("../data/pdfs/annual_report.pdf")
pages = loader.load()    # one Document per page, 0-indexed

print(pages[0].metadata)
# {"source": "../data/pdfs/annual_report.pdf", "page": 0}
```

> **Definition — PyPDFLoader page indexing**
>
> PyPDFLoader uses zero-based page indexing: the first page of a PDF is metadata["page"] == 0. PyMuPDFLoader uses one-based indexing. This asymmetry is one of the most common sources of off-by-one errors when switching between the two loaders. Normalise to a consistent convention during the metadata enrichment pass described in Chapter 4, Section 4.3.

PyMuPDFLoader — C-backed, high fidelity

PyMuPDFLoader is built on PyMuPDF (also known by its import name fitz), a Python binding over the MuPDF C library. MuPDF is the same engine used in many commercial PDF viewers, and its fidelity on complex documents is substantially higher than pypdf. It correctly handles rotated text, preserves whitespace in multi-column layouts more faithfully, and provides richer per-page metadata including the page dimensions, the number of images on the page, and the number of text blocks. The trade-off is the C

dependency: installing pymupdf requires a compiler or a pre-built wheel, which is occasionally problematic in locked-down environments.

```
from langchain_community.document_loaders import PyMuPDFLoader

loader = PyMuPDFLoader("../data/pdfs/annual_report.pdf")
pages = loader.load()   # one Document per page, 1-indexed

print(pages[0].metadata)
# {"source": "annual_report.pdf", "page": 1, "total_pages": 42,
#  "file_path": "../data/pdfs/annual_report.pdf", ...}
```

When to choose which

The decision tree is short. Use PyPDFLoader when your deployment environment prohibits native binaries, when your PDFs are clean and digitally produced, and when install size matters. Use PyMuPDFLoader for everything else — especially when documents contain tables, mixed text and images, rotated content, or unusual fonts. In practice, most production systems settle on PyMuPDFLoader as the default and fall back to PyPDFLoader only when the C dependency cannot be resolved.

Property	PyPDFLoader	PyMuPDFLoader
Underlying library	pypdf (pure Python)	PyMuPDF / fitz (C)
Install complexity	pip only, no compiler	Needs pre-built wheel or compiler
Page indexing	0-based	1-based
Layout fidelity	Basic	High — columns, rotations, tables
Metadata richness	source, page	source, page, total_pages, dimensions, …
Scanned PDF support	None (no OCR)	None (no OCR — see §5.7)
Best for	Simple PDFs, locked envs	Complex PDFs, production systems

5.3 Directory Loading with DirectoryLoader and Glob Patterns

A real knowledge base is not a single file. It is a directory tree — sometimes deeply nested — containing files of many different types. Calling a loader manually on each file is impractical the moment the corpus exceeds a dozen files. DirectoryLoader solves this by accepting a root path, a glob pattern that selects which files to load, and a loader class to use on each matched file.

> **Analogy — The mail room sorter**
>
> Think of DirectoryLoader as a mail room employee whose job is to sort incoming parcels by the label on the front. The glob pattern is the routing rule ("take everything in the pdfs folder with a .pdf sticker"). The sub-loader is the specialist equipment that opens each parcel and reads what is inside. The mail

> room employee does not read the content — that is the specialist's job. The employee just routes, deduplicates, and hands off.

```
from langchain_community.document_loaders import DirectoryLoader,
PyMuPDFLoader

# Load all PDFs recursively under data/pdfs/
loader = DirectoryLoader(
    path="../data/pdfs",
    glob="**/*.pdf",         # ** means recurse into sub-directories
    loader_cls=PyMuPDFLoader,
    use_multithreading=True,  # load files in parallel
    show_progress=True,
)
docs = loader.load()
print(f"Loaded {len(docs)} document pages")
```

Glob pattern vocabulary

Glob patterns follow the standard Unix convention. The single asterisk (*) matches any sequence of characters within a single directory level. The double asterisk (**) matches across directory levels — it is the recursive wildcard. To load all text files in a directory tree you write **/*.txt. To load only PDFs in the root level (not subdirectories) you write *.pdf. Combining multiple extensions in a single DirectoryLoader call is not directly supported; for multi-format directories, use the file-discovery helper in Section 5.8.

Multithreading and silent failures

DirectoryLoader's use_multithreading=True flag loads files in parallel using a thread pool. For large corpora this can cut wall-clock loading time by 4x or more. The catch is silent_errors: when set to True (not the default), files that fail to parse are skipped rather than raising an exception. This is often the right production behaviour — a single corrupt PDF should not abort the ingestion of five hundred clean ones — but it means you must check the loader's error log afterwards. The default is silent_errors=False, which halts on the first failure and is the safer setting during development.

> **Common pitfall — Glob patterns that match nothing**
>
> DirectoryLoader raises no error when the glob pattern matches zero files. It silently returns an empty list. If your pipeline produces an empty vector store and you cannot figure out why, the first thing to check is whether the glob pattern is correct. Print loader.load() on a small test directory first, or use Path(root).glob(pattern) directly in a Python REPL to verify the pattern before wiring it into the pipeline.

5.4 The UnstructuredLoader — One Loader, Many Formats

The Unstructured library, and its LangChain integration langchain-unstructured, takes a different approach to document loading: instead of exposing a separate loader class for each file format, it exposes a single loader that auto-detects the format of each file and dispatches to the appropriate parser internally. One call

to UnstructuredLoader can process a list of files containing PDFs, DOCX files, HTML pages, and plain text, and return a uniform list of Documents from all of them.

> **Definition — UnstructuredLoader**
> A LangChain loader that wraps the open-source Unstructured library. It accepts one or more file paths (or a single path), auto-detects the file type using MIME type sniffing and extension inspection, delegates to the appropriate internal parser, and returns a list of Document objects. It supports over twenty file formats natively, including PDF, DOCX, PPTX, XLSX, HTML, Markdown, CSV, EML, and images with embedded OCR.

```
from langchain_unstructured import UnstructuredLoader

# Pass a mixed list of file paths — Unstructured handles each automatically
file_paths = [
    "../data/pdfs/contract.pdf",
    "../data/docs/policy.docx",
    "../data/web/faq.html",
    "../data/textfiles/notes.txt",
]

loader = UnstructuredLoader(file_paths)
docs = loader.load()

for doc in docs[:3]:
    print(doc.metadata["source"], "->", doc.page_content[:80])
```

The value of UnstructuredLoader is breadth. When your corpus contains many file types and you do not want to maintain a registry of format-specific loaders, UnstructuredLoader reduces the loading layer to a single class and a list of paths. The cost is performance: because it must detect the type of each file before parsing it, and because some of its internal parsers are slower than their specialised equivalents, it is not the right choice for homogeneous corpora of thousands of PDFs where PyMuPDFLoader would be faster.

Element-level vs page-level output

By default, UnstructuredLoader returns one Document per "element" — where an element is a structural unit identified by Unstructured's layout analysis: a heading, a paragraph, a table, a list item. This produces more Documents than page-level loaders, each with richer semantic granularity. If you prefer page-level output (to match the behaviour of PyPDFLoader), set mode="single" for one Document per file or mode="paged" for one Document per page.

```
# Page-level output: one Document per page (matches PyPDFLoader behaviour)
loader = UnstructuredLoader(file_paths, mode="paged")

# Single-document output: entire file as one Document
loader = UnstructuredLoader(file_paths, mode="single")
```

> **Common pitfall — Missing format extras**

UnstructuredLoader installs cleanly from pip but silently falls back to a minimal parser for formats whose extra dependencies are not installed. Attempting to load DOCX files without python-docx installed, or PDFs without pdfminer installed, will produce either empty documents or partial output with no error. The correct installation command is: pip install "unstructured[pdf,docx,xlsx,pptx,html]". Install the extras for every format you intend to load.

5.5 Tabular Data: CSV, Excel, TSV — Converting to JSON for Uniform Ingestion

Tabular data presents a loading problem that has no perfect solution, because the mismatch between how tabular data is structured and how a RAG retriever works is architectural, not incidental. A retriever finds semantically similar passages of text. A spreadsheet row is not a passage — it is a tuple of typed values. The nearest a loader can come to bridging this gap is to serialise each row into a string that a language model can read.

Analogy — The spreadsheet as a filing cabinet

Think of a spreadsheet row as a single folder in a filing cabinet. Each folder has a label (the row's identifying columns) and a set of contents (the remaining columns). When you load the spreadsheet into a RAG system, you are not filing the cabinet — you are photocopying every folder and handing the copies to a librarian who can only read prose. The librarian cannot open the drawer and count the folders. Your job is to write a label on each photocopy that includes everything the librarian might want to know: "Customer ID 4471, Name: Asha Patel, Region: North, Outstanding balance: £2,340."

LangChain's CSVLoader serialises each row to a key: value string, one line per column, separated by newlines. The result is readable but verbose. A better approach — used in production systems where CSV rows contain many columns — is to convert each row to a compact JSON string and store it as the page_content. JSON is more compact than the CSVLoader default, is parseable by language models, and preserves the data types of numeric columns.

CSVLoader — the built-in approach

```
from langchain_community.document_loaders.csv_loader import CSVLoader

# Each row becomes one Document
loader = CSVLoader(
    file_path="../data/csv/customers.csv",
    csv_args={"delimiter": ","},
    source_column="customer_id",   # use this column as the metadata source
)
docs = loader.load()
print(docs[0].page_content)
# customer_id: 4471
# name: Asha Patel
# region: North
# balance: 2340
```

JSON serialisation — the production approach

```
import json, pandas as pd
from langchain_core.documents import Document

def csv_to_documents(path: str) -> list[Document]:
    df = pd.read_csv(path)
    docs = []
    for _, row in df.iterrows():
        content = json.dumps(row.to_dict(), ensure_ascii=False)
        docs.append(Document(
            page_content=content,
            metadata={"source": path, "row_index": int(_)},
        ))
    return docs

# Works identically for Excel and TSV:
def excel_to_documents(path: str, sheet: str = None) -> list[Document]:
    df = pd.read_excel(path, sheet_name=sheet or 0)
    return [
        Document(
            page_content=json.dumps(row.to_dict(), ensure_ascii=False),
            metadata={"source": path, "sheet": sheet or "Sheet1", "row":
i},
        )
        for i, (_, row) in enumerate(df.iterrows())
    ]
```

The JSON approach gives you one additional benefit: it is trivially extensible to TSV files (pd.read_csv with sep="\t"), Excel workbooks with multiple sheets, and any other Pandas-readable format, all sharing the same serialisation logic. Chapter 6 revisits tabular data from the other direction — explaining when CSV content should not be in a RAG system at all, and when a two-track architecture is the correct response.

5.6 HTML, Markdown, DOCX, and JSON

These four formats each present a different extraction challenge. HTML is the richest in non-content noise — navigation bars, footers, advertisement slots, and JavaScript fragments all appear in the raw file alongside the actual text. Markdown is the cleanest — its structural markers are minimal and meaningful. DOCX carries semantic structure (headings, tables, lists) that is worth preserving. JSON is either document-like (a nested object with prose values) or record-like (an array of rows), and the two cases require completely different loading strategies.

HTML with BSHTMLLoader

BSHTMLLoader uses BeautifulSoup to strip HTML tags and extract the text content of the page. It returns one Document per file, with the cleaned text as page_content and the HTML title element (if present) copied into metadata["title"]. The extractor discards scripts, styles, and navigation elements by default.

```
from langchain_community.document_loaders import BSHTMLLoader

loader = BSHTMLLoader("../data/web/product_faq.html")
docs = loader.load()
print(docs[0].metadata)   # {"source": "...", "title": "Product FAQ"}
```

Markdown with UnstructuredMarkdownLoader

Markdown files can be loaded either as plain text (using TextLoader, which ignores the markup) or as structured documents (using UnstructuredMarkdownLoader, which parses headings and returns one element per heading section). The structured approach is almost always preferable: it preserves heading hierarchy in metadata and produces more semantically coherent chunks, because each chunk stays within its section boundary.

```
from langchain_community.document_loaders import UnstructuredMarkdownLoader

loader = UnstructuredMarkdownLoader("../data/docs/README.md",
mode="elements")
elements = loader.load()
# Each element is one heading + its body text
for el in elements[:2]:
    print(el.metadata.get("category"), "->", el.page_content[:60])
```

DOCX with Docx2txtLoader

Word documents (.docx) are ZIP archives containing XML. LangChain's Docx2txtLoader extracts the plain text from the body of the document and returns a single Document. Tables are linearised to rows of tab-separated values. For documents where table structure matters — policy documents with structured clauses, contracts with term sheets — UnstructuredLoader with mode="elements" provides richer output by returning tables as separate elements with their structure annotated in metadata.

```
from langchain_community.document_loaders import Docx2txtLoader
```

```
loader = Docx2txtLoader("../data/docs/sla_contract.docx")
docs = loader.load()   # one Document, full text

# For structured DOCX (tables preserved as elements):
from langchain_unstructured import UnstructuredLoader
loader = UnstructuredLoader("../data/docs/sla_contract.docx",
mode="elements")
elements = loader.load()
```

JSON with JSONLoader

JSON files require a jq-style path expression that tells the loader which field in the JSON structure contains the text to extract. Without this, the loader cannot know whether the page_content should be the "body" key, the "content" key, the "text" key, or a concatenation of several fields. The jq_schema parameter accepts a standard jq path: .[] extracts each element of a top-level array; .articles[].body extracts the body field from each element of an articles array.

```
from langchain_community.document_loaders import JSONLoader

# JSON file: [{"id": 1, "title": "...", "body": "..."}, ...]
loader = JSONLoader(
    file_path="../data/json/articles.json",
    jq_schema=".[]",           # iterate over the top-level array
    content_key="body",        # use the "body" field as page_content
    metadata_func=lambda rec, _: {"source": "articles.json", "id":
rec["id"]},
)
docs = loader.load()
```

5.7 Handling Messy and Scanned PDFs (OCR Fallbacks)

Not all PDFs contain extractable text. Scanned documents — contracts photographed on a flatbed scanner, printed reports digitised by a document management system, archived letters converted to PDF by a copier — store their content as rasterised page images. A text extractor operating on these files returns an empty string, not an error. The document loads without complaint; the page_content is blank; the system ingests nothing of value. OCR (Optical Character Recognition) is the corrective.

> **Definition — OCR (Optical Character Recognition)**
> A computational process that converts raster images of text — photographs, scans, or bitmap PDFs — into machine-readable strings. Modern OCR systems apply convolutional neural networks to detect character bounding boxes, then classify each region into a character. The output is an approximate text reconstruction: accuracy is typically 95–99% on clean printed documents, lower on handwritten text, degraded scans, or unusual fonts.

Detecting when OCR is needed

Before applying OCR, you need to know whether a PDF actually needs it. The diagnostic is simple: load the PDF with PyMuPDFLoader, check whether page_content is empty (or suspiciously short) on pages that visually appear to contain text, and flag those pages for OCR. A content length threshold of fewer than 30 characters per page is a reliable heuristic — a genuine blank page has fewer than five words, so anything in between is almost certainly a failed text extraction.

```
from langchain_community.document_loaders import PyMuPDFLoader

def needs_ocr(pdf_path: str, threshold: int = 30) -> bool:
    """Return True if the PDF appears to be a scan (no extractable
text)."""
    docs = PyMuPDFLoader(pdf_path).load()
    avg_len = sum(len(d.page_content) for d in docs) / max(len(docs), 1)
    return avg_len < threshold
```

OCR with pytesseract and pdf2image

The most common open-source OCR pipeline for Python uses pdf2image to rasterise each PDF page to a PIL Image, then pytesseract (a Python wrapper over the Tesseract OCR engine) to extract text from each image. The result is assembled into Document objects with the same metadata structure as a standard PDF loader.

```
# pip install pdf2image pytesseract
# System: apt install tesseract-ocr poppler-utils  (on Debian/Ubuntu)

from pdf2image import convert_from_path
import pytesseract
from langchain_core.documents import Document

def ocr_pdf(pdf_path: str, dpi: int = 300) -> list[Document]:
    """Rasterise each page and OCR it into a Document."""
    images = convert_from_path(pdf_path, dpi=dpi)
    docs = []
    for i, image in enumerate(images):
        text = pytesseract.image_to_string(image, lang="eng")
        docs.append(Document(
            page_content=text,
            metadata={"source": pdf_path, "page": i + 1, "ocr": True},
        ))
    return docs
```

The ocr=True metadata flag is important: it marks OCR-derived content so that downstream components — the self-learning loop in Part IV, the quality evaluator in Chapter 11 — can apply lower confidence to answers derived from OCR text and prioritise re-checking against the original document when possible.

The unified fallback pattern

In production pipelines, OCR should be a fallback, not the default. The correct pattern is: try the standard PDF loader first; if the extracted text is below the diagnostic threshold, re-load with OCR. This keeps the OCR path (which is 10–100x slower) out of the critical path for clean documents.

```
def load_pdf_with_fallback(pdf_path: str) -> list[Document]:
    """Try standard extraction; fall back to OCR if text is absent."""
    docs = PyMuPDFLoader(pdf_path).load()
    avg_len = sum(len(d.page_content) for d in docs) / max(len(docs), 1)
    if avg_len < 30:
        print(f"[OCR fallback] {pdf_path}")
        docs = ocr_pdf(pdf_path)
    return docs
```

Common pitfall — OCR on already-extractable PDFs

Running OCR on a PDF that already contains a text layer produces degraded output. The rasterisation step introduces compression artefacts; Tesseract then misreads characters that the text extractor would have returned perfectly. Always apply the needs_ocr diagnostic before invoking the OCR path. The DPI parameter to convert_from_path also matters: 300 DPI is the correct default for printed business documents; 150 DPI is acceptable for pre-screening, but will produce noticeably more character errors on small fonts.

5.8 Building a Unified File-Discovery Helper Across Multiple Data Roots

The sections above covered each loader family in isolation. A production ingestion pipeline needs to combine them: walk an arbitrary set of directories, deduplicate files that appear under multiple roots (a common occurrence with symlinked or mirrored storage), select the correct loader for each file based on its extension, and return a single, ordered list of Documents ready for the chunker. This section builds that helper.

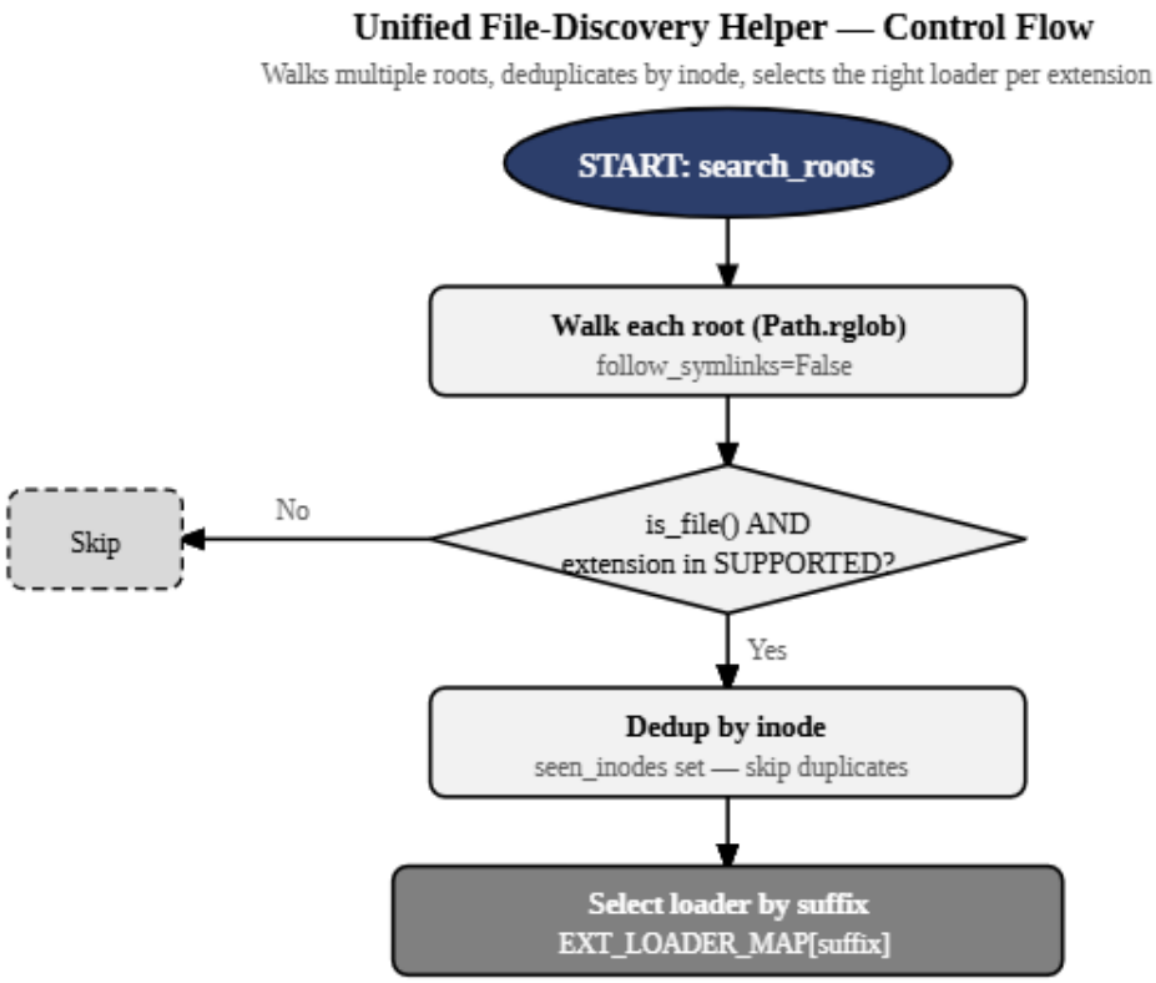

Figure 5.2 — Unified file-discovery helper control flow. The helper walks each root, filters by supported extension, deduplicates by inode, selects the correct loader, and returns a flat list of Documents.

The extension-to-loader registry

The registry is a dictionary that maps file extensions to loader factories — callables that accept a path and return a list of Documents. Keeping the registry explicit rather than embedding the dispatch logic in a long if-elif chain has two advantages: the registry is easy to extend (add one line to support a new format), and it is easy to inspect in a REPL or test.

```
from pathlib import Path
from langchain_community.document_loaders import (
    TextLoader, PyMuPDFLoader, BSHTMLLoader,
    Docx2txtLoader, DirectoryLoader,
)
from langchain_unstructured import UnstructuredLoader

def _make_text(path):   return TextLoader(path, encoding="utf-8").load()
def _make_pdf(path):    return load_pdf_with_fallback(path)
def _make_html(path):   return BSHTMLLoader(path).load()
def _make_docx(path):   return Docx2txtLoader(path).load()
def _make_unstruc(path): return UnstructuredLoader(path).load()

EXT_LOADER_MAP = {
    ".txt":  _make_text,
    ".md":   _make_text,
    ".log":  _make_text,
    ".pdf":  _make_pdf,
    ".html": _make_html,
    ".htm":  _make_html,
    ".docx": _make_docx,
    ".pptx": _make_unstruc,
    ".csv":  lambda p: csv_to_documents(p),
    ".tsv":  lambda p: csv_to_documents(p),
    ".xlsx": lambda p: excel_to_documents(p),
}
```

The discovery loop

The loop uses pathlib's rglob method to traverse each root recursively. It deduplicates files using their OS inode number — a unique identifier that remains stable across symbolic links — so that the same physical file reached through two different paths is loaded exactly once. The inode approach is more robust than path-string comparison, which would treat /data/pdfs/contract.pdf and /archive/../data/pdfs/contract.pdf as different files.

```
def discover_and_load(
    roots: list[str | Path],
    extensions: set[str] | None = None,
    verbose: bool = True,
) -> list:
    """Walk roots, deduplicate, and load all supported files."""
    if extensions is None:
        extensions = set(EXT_LOADER_MAP.keys())

    seen_inodes: set[int] = set()
    all_docs = []

    for root in roots:
        root = Path(root)
        if not root.exists():
            print(f"[warn] Root does not exist: {root}")
            continue
        for p in root.rglob("*"):
            if not p.is_file():
                continue
            if p.suffix.lower() not in extensions:
                continue
            inode = p.stat().st_ino
            if inode in seen_inodes:
                continue    # skip duplicate
            seen_inodes.add(inode)
            loader_fn = EXT_LOADER_MAP.get(p.suffix.lower())
            if loader_fn is None:
                continue
            try:
                docs = loader_fn(str(p))
                all_docs.extend(docs)
                if verbose:
                    print(f"  [OK] {p.name}: {len(docs)} doc(s)")
            except Exception as exc:
                print(f"  [FAIL] {p.name}: {exc}")

    return all_docs
```

Wiring it into the pipeline

```
# Usage: point at as many roots as you like
search_roots = [
```

```
    Path("../data/pdfs"),
    Path("../data/textfiles"),
    Path("../data/csv"),
    Path("../data/docs"),
]

all_documents = discover_and_load(search_roots, verbose=True)
print(f"\nTotal documents loaded: {len(all_documents)}")

# Hand off to the chunker (Chapter 7)
# chunks = split_documents(all_documents)
```

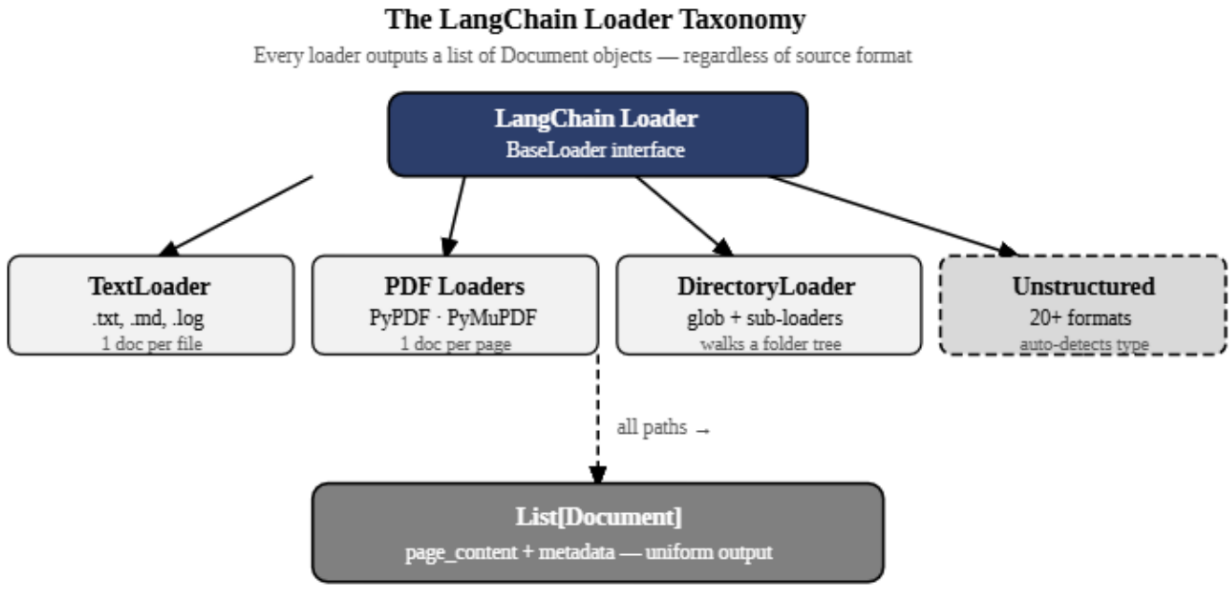

Figure 5.3 — The LangChain loader taxonomy. Every loader, regardless of format, produces the same output: a list of Document objects with page_content and metadata.

The helper above is intentionally minimal. Three extensions make it production-ready. First, add a max_file_size_bytes parameter and skip files above the threshold — a 2 GB CAD drawing has no business in a text-based RAG index. Second, add a last_modified_after parameter using the file_facts function from Chapter 4, Layer 2, so incremental ingestion runs only pick up files changed since the last run. Third, route CSV and Excel files to the CSV-track analyser described in Chapter 6 rather than embedding their rows directly, unless the domain analysis in Chapter 6 determines that row-level retrieval is appropriate for those specific files.

> **Analogy — The universal adapter plug**
>
> The discover_and_load helper is the universal travel adapter for your data pipeline. You plug it into any wall socket in the world — any directory structure, any mix of file types — and it delivers the same clean current: a Python list of Document objects. The individual loaders are the adapter's internal circuitry, each handling a different voltage. You never change the adapter; you change which socket you plug it into.

This chapter completes the loading layer of the ingestion pipeline. You now have a loader for every file type your data team is likely to hand you, a fallback strategy for the hardest case (scanned PDFs), and a unified helper that abstracts the entire loading layer behind a single function call. The next chapter breaks the assumption that all documents should go into the same RAG system. Chapter 6 — When RAG Is the Wrong Tool: The Two-Track Problem — shows you how to detect when a user's question requires aggregation over structured data rather than semantic retrieval over prose, and how to build a pipeline that routes the two types correctly.